

Sprawozdanie z Laboratorium nr 4

Przedmiot: Integracja Systemów Informatycznych **Temat:** Konteneryzacja aplikacji Django za pomocą Dockera **Student:** Anna Godlewska **Data:** 05.05.2026

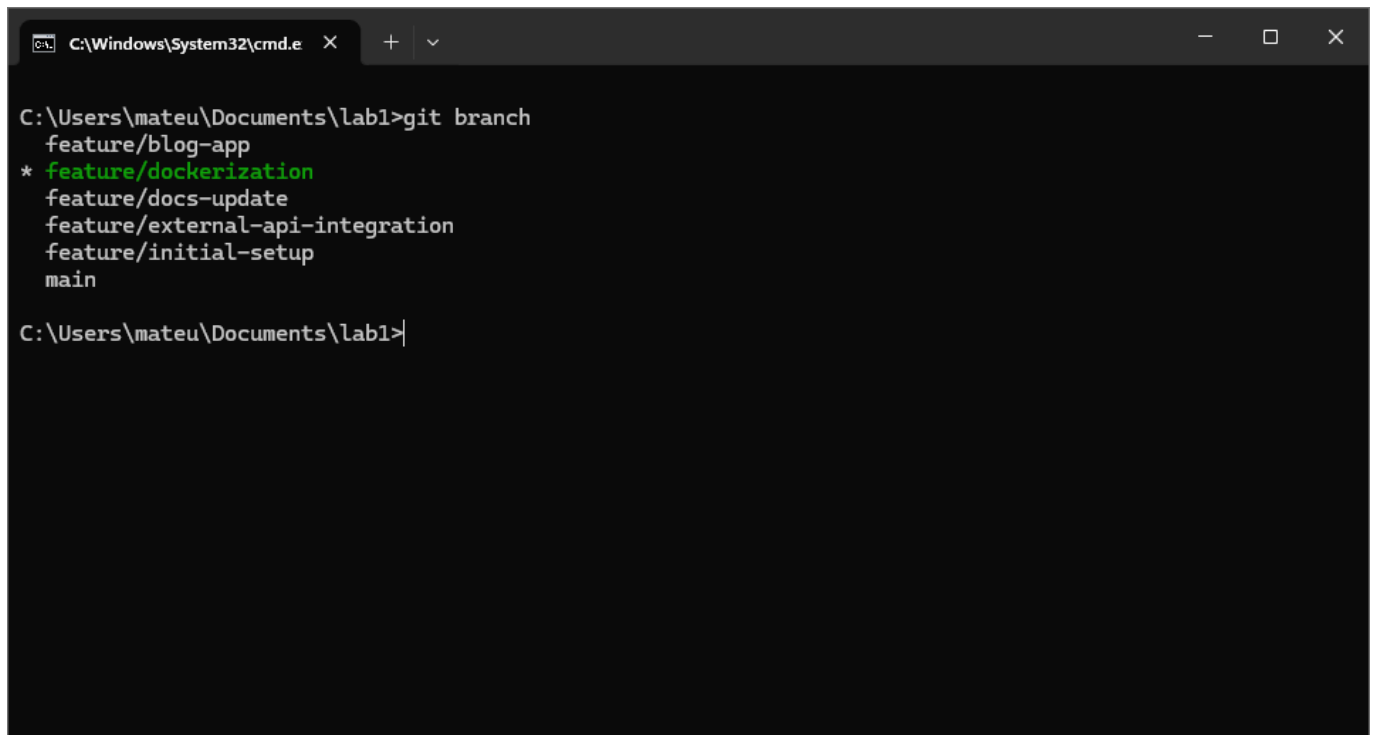
1. Cel laboratorium

Celem laboratorium było przygotowanie aplikacji (w tym przypadku stworzonego wcześniej bloga) do pracy w środowisku izolowanym przy użyciu Dockera. Skonfigurowano plik `Dockerfile` dla aplikacji Django oraz stworzono orkiestrację z bazą danych PostgreSQL za pomocą pliku `docker-compose.yml`.

2. Realizacja zadań

Zadanie 1: Przygotowanie środowiska i pliki konfiguracyjne

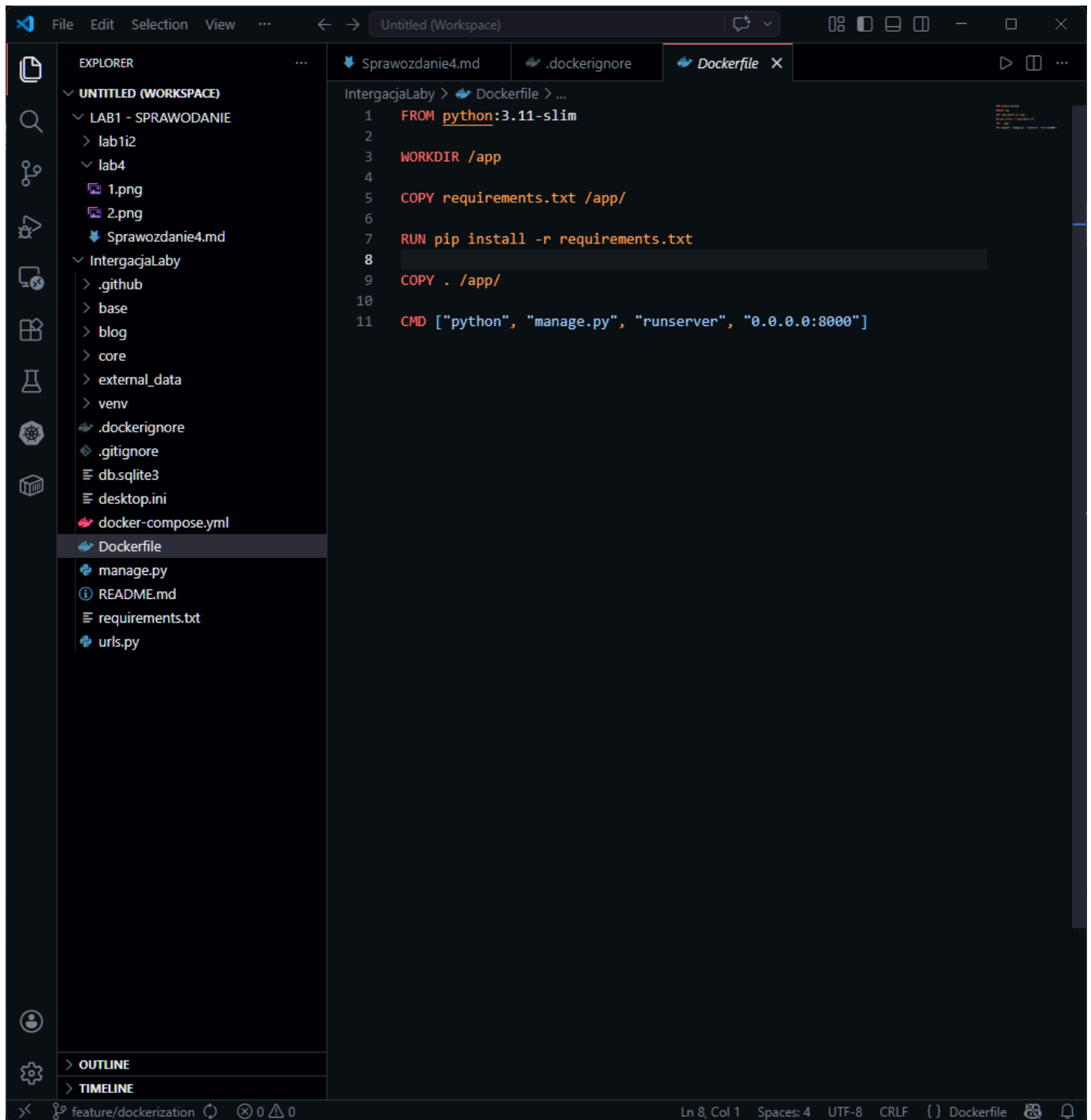
Prace rozpoczęto od utworzenia dedykowanej gałęzi `feature/dockerization`. Następnie utworzono plik `.dockerignore`, aby zapobiec kopiowaniu zbędnych elementów do obrazu (takich jak katalog środowiska wirtualnego `venv/` czy pliki `__pycache__`). Zadbano o kompletność pliku `requirements.txt` dla projektu w języku Python.



```
C:\Windows\System32\cmd.e  X  +  v  -  □  X

C:\Users\mateu\Documents\lab1>git branch
feature/blog-app
* feature/dockerization
feature/docs-update
feature/external-api-integration
feature/initial-setup
main

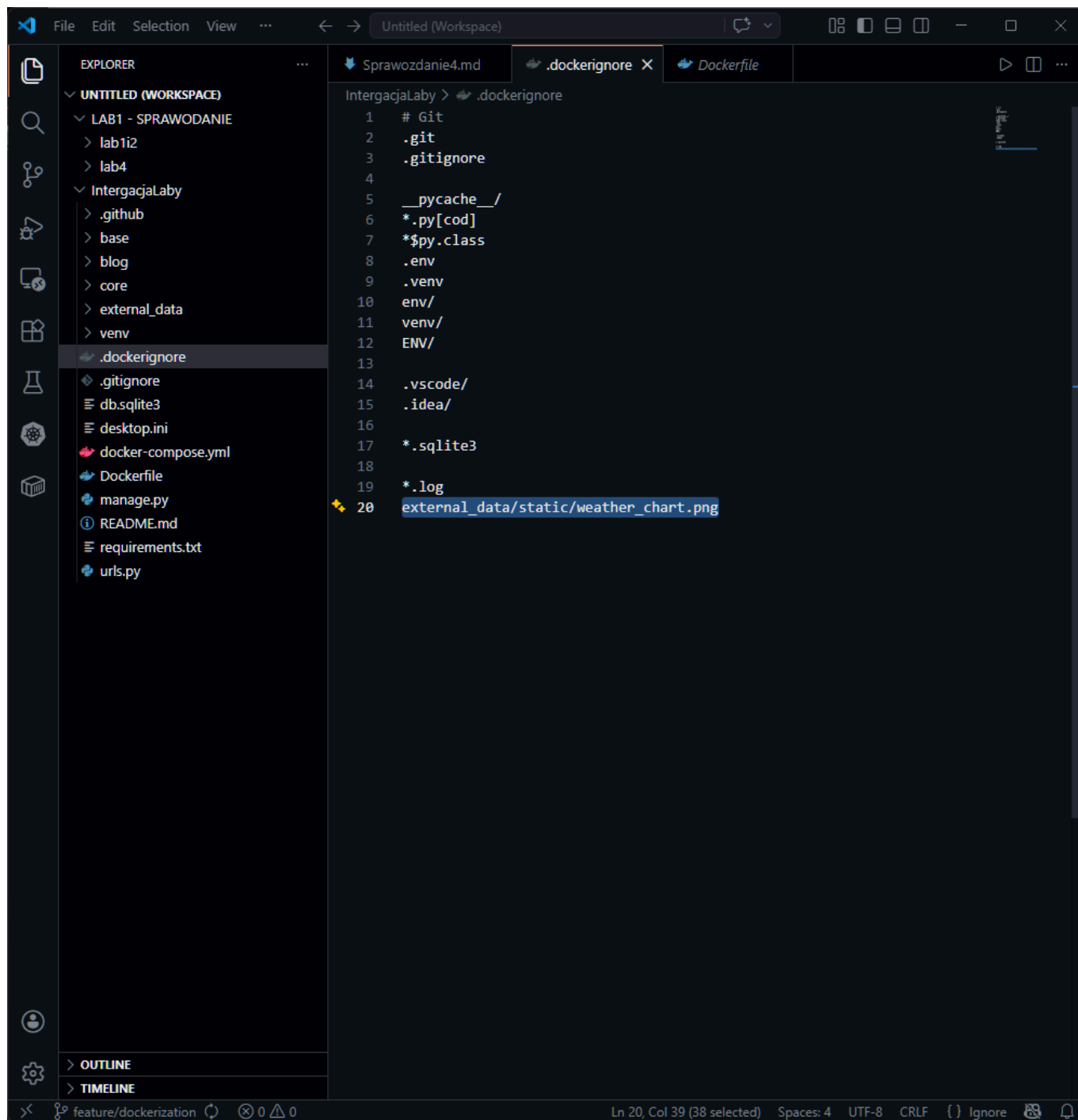
C:\Users\mateu\Documents\lab1>
```



Zadanie 2: Tworzenie i budowanie obrazu Docker (Dockerfile)

Przygotowano plik **Dockerfile** oparty na rekomendowanym, odchudzonym obrazie bazowym **python:3.11-slim**. W pliku zdefiniowano proces kopiowania plików, instalacji pakietów za pomocą **pip install -r requirements.txt** oraz ustawiono domyślną komendę startową (**CMD**) uruchamiającą serwer **python manage.py runserver 0.0.0.0:8000**.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt /app/
RUN pip install -r requirements.txt
COPY . /app/
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left and the Editor view on the right. The Explorer sidebar shows a project structure with folders like 'lab1i2', 'lab4', and 'IntegracjaLaby'. The 'IntegracjaLaby' folder is expanded, showing files like '.github', 'base', 'blog', 'core', 'external_data', 'venv', '.dockerignore', '.gitignore', 'db.sqlite3', 'desktop.ini', 'docker-compose.yml', 'Dockerfile', 'manage.py', 'README.md', 'requirements.txt', and 'urls.py'. The '.dockerignore' file is selected and its content is displayed in the Editor view. The content of the '.dockerignore' file is as follows:

```
1 # Git
2 .git
3 .gitignore
4
5 __pycache__/
6 *.py[cod]
7 *$py.class
8 .env
9 .venv
10 env/
11 venv/
12 ENV/
13
14 .vscode/
15 .idea/
16
17 *.sqlite3
18
19 *.log
20 external_data/static/weather_chart.png
```

The status bar at the bottom indicates the current file is 'feature/dockerization', the cursor is at 'Ln 20, Col 39 (38 selected)', and the encoding is 'UTF-8'.

Zbudowano obraz wpisując polecenie `docker build -t my-app .` i zatwierdzono te zmiany pierwszym commitem ("Add Dockerfile and .dockerignore").

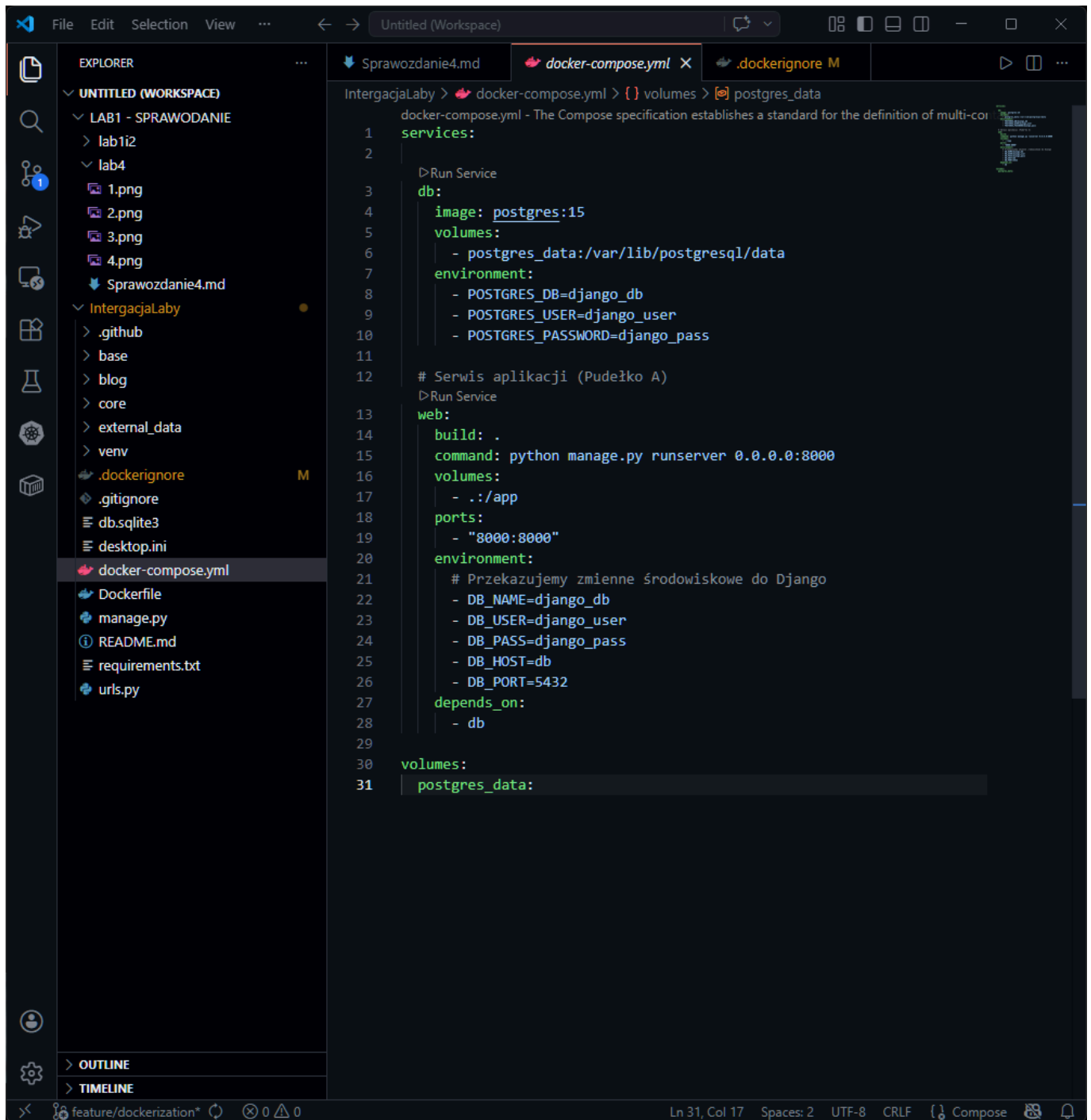
```
C:\Windows\System32\cmd.e X + v
C:\Users\mateu\Documents\IntergacjaLaby>docker images

Info → U In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
hello-world:latest f9078146db2e 25.9kB 9.49kB U
intergacjalaby-web:latest 82356e9078b9 622MB 170MB U
lab1-web:latest fa4be0584961 622MB 170MB
my-app:latest 14bbf8f70492 622MB 170MB
postgres:15 29342cb52157 633MB 164MB U

C:\Users\mateu\Documents\IntergacjaLaby>
```

Zadanie 3: Orkiestracja z Docker Compose

Stworzono plik `docker-compose.yml`, który definiuje i integruje dwa serwisy: `web` (kontener z aplikacją Django) i `db` (kontener bazy PostgreSQL). Skonfigurowano zmienne środowiskowe z uprawnieniami (user, hasło, nazwa bazy) oraz zdefiniowano wolumen dla bazy, zapewniając trwałość danych. Wprowadzono zmiany w repozytorium commitem "Add docker-compose for app and database orchestration".



The screenshot shows the Visual Studio Code interface with a workspace named 'Untitled (Workspace)'. The Explorer sidebar on the left shows a project structure for 'IntegracjaLaby' with files like 'Dockerfile', 'manage.py', 'README.md', 'requirements.txt', and 'urls.py'. The main editor area displays the 'docker-compose.yml' file, which defines two services: 'db' (PostgreSQL) and 'web' (Django application). The 'db' service uses the 'postgres:15' image and maps a volume 'postgres_data' to '/var/lib/postgresql/data'. It also sets environment variables for database name, user, password, and host. The 'web' service builds from the current directory and runs 'python manage.py runserver 0.0.0.0:8000'. It maps the local directory to the container and sets environment variables for database connection. The 'web' service depends on the 'db' service. A volume 'postgres_data' is defined at the bottom.

```
1 services:
2
3     >Run Service
4     db:
5         image: postgres:15
6         volumes:
7             - postgres_data:/var/lib/postgresql/data
8         environment:
9             - POSTGRES_DB=django_db
10            - POSTGRES_USER=django_user
11            - POSTGRES_PASSWORD=django_pass
12
13     # Serwis aplikacji (Pudełko A)
14     >Run Service
15     web:
16         build: .
17         command: python manage.py runserver 0.0.0.0:8000
18         volumes:
19             - ../app
20         ports:
21             - "8000:8000"
22         environment:
23             # Przekazujemy zmienne środowiskowe do Django
24             - DB_NAME=django_db
25             - DB_USER=django_user
26             - DB_PASS=django_pass
27             - DB_HOST=db
28             - DB_PORT=5432
29         depends_on:
30             - db
31
32 volumes:
33     postgres_data:
```

Zadanie 4: Połączenie z bazą danych i Migracje

Przed pełnym wdrożeniem zmodyfikowano kod aplikacji. Aby umożliwić nawiązanie połączenia, w pliku konfiguracyjnym aplikacji zmieniono adres hosta bazy danych z domyślnego na wyznaczony w ustawieniach Compose adres serwisu (**db**).

```
IntegracjaLaby > core > settings.py > ...
62
63     'context_processors': [
64         'django.template.context_processors.request',
65         'django.contrib.auth.context_processors.auth',
66         'django.contrib.messages.context_processors.messages',
67     ],
68 ],
69 },
70 ],
71
72 WSGI_APPLICATION = 'core.wsgi.application'
73
74
75 # Database
76 # https://docs.djangoproject.com/en/6.0/ref/settings/#databases
77
78 DATABASES = {
79     'default': {
80         'ENGINE': 'django.db.backends.postgresql',
81         'NAME': os.environ.get('DB_NAME'),
82         'USER': os.environ.get('DB_USER'),
83         'PASSWORD': os.environ.get('DB_PASS'),
84         'HOST': os.environ.get('DB_HOST'),
85         'PORT': os.environ.get('DB_PORT', '5432'),
86     }
87 }
88
89
90 # Password validation
91 # https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators
92
93 AUTH_PASSWORD_VALIDATORS = [
94     {
95         'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
96     },
97     {
98         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
99     },
100     {
101         'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
102     },
103     {
104         'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
105     },
106 ]
107
108
109 # Internationalization
110 # https://docs.djangoproject.com/en/6.0/topics/i18n/
```

Po uruchomieniu zestawu kontenerów, konieczne było utworzenie tabel. Będąc połączonym z powłoką kontenera, przeprowadzono proces tworzenia struktury bazy danych.

```
C:\Windows\System32\cmd.e X + v
C:\Users\mateu\Documents\IntergacjaLaby>docker-compose exec web python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, blog, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying blog.0001_initial... OK
  Applying sessions.0001_initial... OK

C:\Users\mateu\Documents\IntergacjaLaby>
C:\Users\mateu\Documents\IntergacjaLaby>
C:\Users\mateu\Documents\IntergacjaLaby>
C:\Users\mateu\Documents\IntergacjaLaby>
C:\Users\mateu\Documents\IntergacjaLaby>
```

Zadanie 5: Zarządzanie kontenerami i weryfikacja działania

Cały stos technologiczny uruchomiono pomyślnie używając narzędzia do orkiestracji (za pomocą polecenia `docker-compose up -d`). Skontrolowano funkcjonowanie i stabilność środowiska uruchamiając komendy `docker ps` oraz logując się do wewnętrznego terminala serwisu (`docker-compose exec web bash`).


```
C:\Windows\System32\cmd.e X + v
C:\Users\mateu\Documents\IntergacjaLaby>docker-compose up -d
[+] up 2/2
✓Container intergacjalaby-db-1 Running 0.0s
✓Container intergacjalaby-web-1 Running 0.0s

C:\Users\mateu\Documents\IntergacjaLaby>docker-compose logs -f web
web-1 | Watching for file changes with StatReloader
web-1 | /app/core/settings.py changed, reloading.
web-1 | Performing system checks...
web-1 |
web-1 | System check identified no issues (0 silenced).
web-1 |
web-1 | You have 19 unapplied migration(s). Your project may not work properly until you ap
ply the migrations for app(s): admin, auth, blog, contenttypes, sessions.
web-1 | Run 'python manage.py migrate' to apply them.
web-1 | May 05, 2026 - 20:38:39
web-1 | Django version 5.2.13, using settings 'core.settings'
web-1 | Starting development server at http://0.0.0.0:8000/
web-1 | Quit the server with CONTROL-C.
web-1 |
web-1 | WARNING: This is a development server. Do not use it in a production setting. Use a
production WSGI or ASGI server instead.
web-1 | For more information on production servers see: https://docs.djangoproject.com/en/5
.2/howto/deployment/
web-1 | Watching for file changes with StatReloader
web-1 | System check identified some issues:
web-1 |
web-1 | WARNINGS:
web-1 | blog.Post: (models.W042) Auto-created primary key used when not defining a primary
```

```

C:\Windows\System32\cmd.e X + v
web-1 | For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
web-1 | Watching for file changes with StatReloader
web-1 | /app/core/settings.py changed, reloading.
web-1 | Performing system checks...
web-1 |
web-1 | System check identified no issues (0 silenced).
web-1 |
web-1 | You have 19 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, blog, contenttypes, sessions.
web-1 | Run 'python manage.py migrate' to apply them.
web-1 | May 05, 2026 - 20:48:46
web-1 | Django version 5.2.13, using settings 'core.settings'
web-1 | Starting development server at http://0.0.0.0:8000/
web-1 | Quit the server with CONTROL-C.
web-1 |
web-1 | WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
web-1 | For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
web-1 | Watching for file changes with StatReloader

C:\Users\mateu\Documents\IntergacjaLaby>docker-compose exec web bash
root@e461aa30832f:/app# exit
exit

C:\Users\mateu\Documents\IntergacjaLaby>docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS
TUS           PORTS
e461aa30832f   intergacjalaby-web                  "python manage.py ru..."  34 minutes ago Up
34 minutes    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp   intergacjalaby-web-1
4e2211238426   postgres:15                         "docker-entrypoint.s..."  34 minutes ago Up
34 minutes    5432/tcp                                           intergacjalaby-db-1

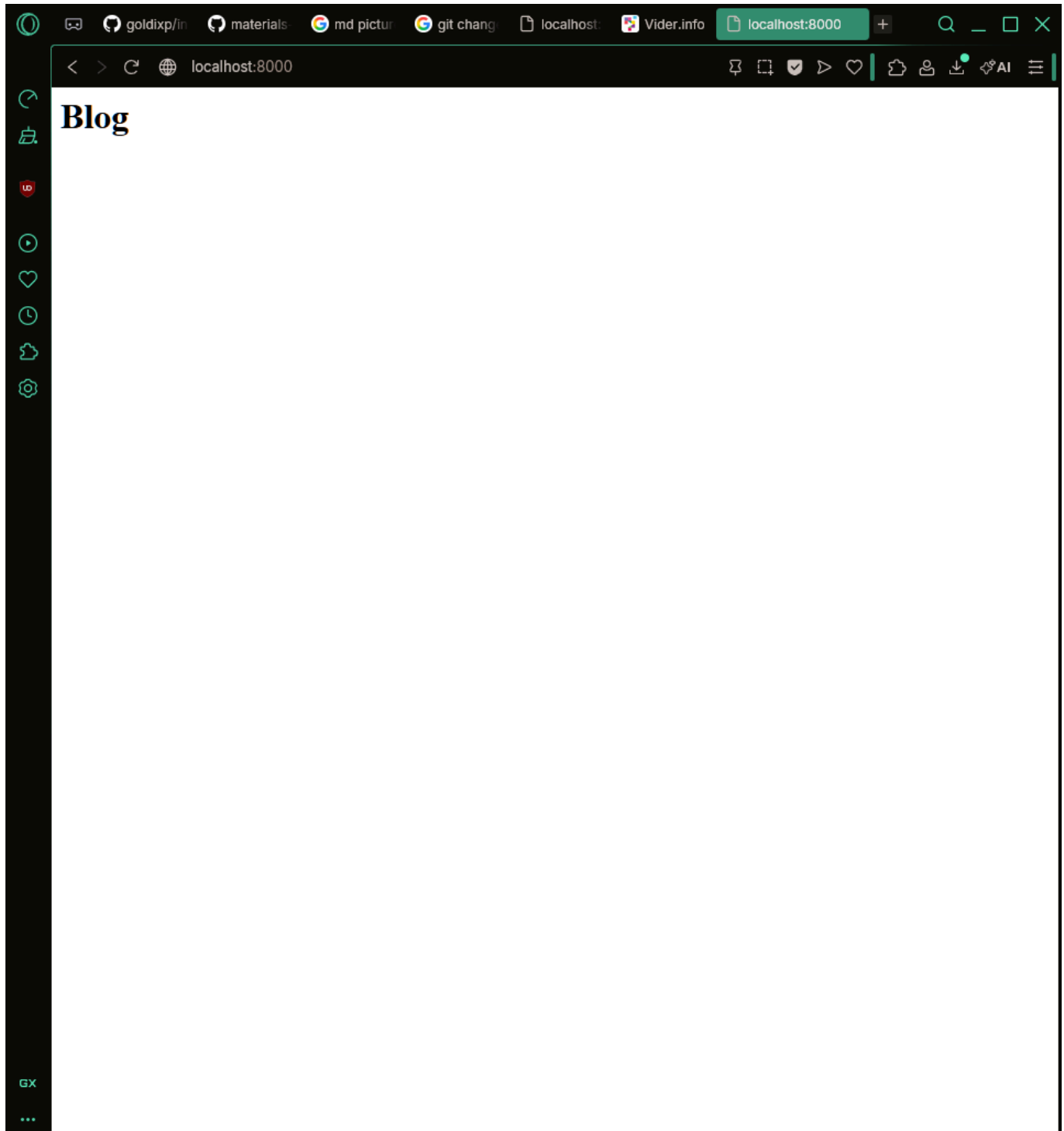
C:\Users\mateu\Documents\IntergacjaLaby>

```

3. Checklist

- ☒ Czy stworzono i wykorzystano nową gałąź `feature/dockerization`?
- ☒ Czy stworzono plik `.dockerignore`, aby zminimalizować rozmiar obrazu?
- ☒ Czy plik zależności (`requirements.txt` lub `package.json`) zawiera wszystkie niezbędne biblioteki?
- ☒ Czy `Dockerfile` bazuje na oficjalnym obrazie odpowiednim dla technologii (`python:slim`, `node:slim`)?
- ☒ Czy `Dockerfile` poprawnie kopiuje pliki źródłowe i instaluje zależności?
- ☒ Czy w `Dockerfile` zdefiniowano odpowiednią komendę startową (`CMD`)?
- ☒ Czy obraz Dockera buduje się bez błędów (`docker build -t app-test .`)?
- ☒ Czy plik `docker-compose.yml` zawiera co najmniej dwa serwisy: `web` (aplikacja) i `db` (baza danych)?
- ☒ Czy zmienne środowiskowe (`host`, `port`, `user`, `password`) są poprawnie przekazywane do aplikacji w `docker-compose.yml`?
- ☒ Czy w kodzie aplikacji (np. `settings.py` lub `.env`) baza danych jest skonfigurowana do łączenia się z nazwą serwisu zdefiniowaną w Compose (np. `db`) zamiast `localhost`?

- ☒ Czy wolumeny (volumes) dla bazy danych są poprawnie zdefiniowane (np. w celu zachowania danych)?
- ☒ Czy sprawdzono logi kontenerów za pomocą `docker-compose logs`?
- ☒ Czy potrafisz wejść do powłoki (shell) działającego kontenera?
- ☒ Czy aplikacja uruchamia się poprawnie za pomocą komendy `docker-compose up` i jest dostępna w przeglądarce?



- ☒ Czy wykonano migracje bazy danych wewnątrz kontenera (np. `docker-compose exec web python manage.py migrate`)?
- ☒ Czy sprawozdanie w formacie PDF zostało przygotowane (zawiera zrzuty ekranu z budowania obrazu i działania kontenerów)?

4. Wnioski

Laboratorium 4 pozwoliło na zdobycie praktycznych umiejętności w zakresie wdrażania rozwiązań opartych na izolowanych środowiskach (kontenerach). Pozwoliło to na zapoznanie się z narzędziem Docker, a przy tym naukę eliminacji problemów ze środowiskiem deweloperskim i kompatybilnością baz danych. Rozwiązanie to gwarantuje przenośność aplikacji na dowolną maszynę z zachowaniem stabilności i odpowiedniej izolacji.